

1. Parcourir un tableau

```
nombres=[1,2,3,4,5,6]
for i in range(len(nombres)):
    print("indice ",i, " : ",nombres[i])
```

```
nombres=[1,2,3,4,5,6]
for element in nombres:
    print("nombre: ",element)
```

i = 0, 1, 2, 3, 4, 5
affiche l'élément à l'indice i

on parcourt la liste sans se préoccuper de la position de l'élément

2. Échanger 2 éléments d'un tableau

```
nombres=[62,24,32,47,58,6]
nombres[0], nombres[5]=nombres[5], nombres[0]
print(nombres)
```

3. Le saucissonnage ou slicing

Tester ces lignes et commentez :

```
liste = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']
liste1=liste[2:5]
liste2=liste[6:2:-1]
liste3=liste[:6]
liste4=liste[2:]
liste5=liste[:]
```

liste1 : éléments à partir de l'indice 2 jusqu'à l'indice 5
liste2 : éléments à partir de l'indice 6 jusqu'à l'indice 2 (à l'envers)
liste3 : éléments à partir de l'indice 0 jusqu'à l'indice 6
liste4 : éléments à partir de l'indice 2 jusqu'à la fin
liste5 : tous les éléments de la liste

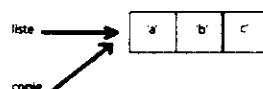
4. Copie de tableaux

Tester ces lignes et commentez :

```
liste = ['a', 'b', 'c']
copie=liste
print(copie)
copie[0]="modifié"
liste[1]="celui-là aussi"
print(liste)
print(copie)
```

la liste copie est une copie de la liste liste
on modifie l'élément à l'indice 0 de la copie
on modifie l'élément à l'indice 1 de la liste
la liste copie est toujours la même

Explications : en fait, une liste en Python ne contient que l'adresse mémoire où sont stockées ses éléments : c'est ce qu'on appelle un pointeur.



Quand on copie liste dans copie, c'est cette adresse mémoire qui est copiée. C'est un alias qui est créé.



Quand on modifie un élément d'une liste, il est alors aussi modifié dans la copie. L'avantage étant qu'on encombre moins la mémoire centrale puisque les éléments de la liste ne figurent qu'en un seul emplacement mémoire.

On peut utiliser la fonction intégrée `id()` qui retourne un entier long étant l'image de l'adresse mémoire où est stocké l'objet passé en argument.
Ou encore l'opérateur `is` qui permet de tester l'égalité des références.
On constate que les 2 objets ont le même identifiant.

```
print(id(liste))
print(id(copie))
print(liste is copie)
```

1^{re} - NSI - FICHE 7.2 - TYPES DE DONNEES CONSTRUIITS LES TABLEAUX

I. DÉFINITION

Un tableau (ou liste en Python) permet de stocker plusieurs valeurs ou données dans une seule variable. Ces valeurs ou données sont appelées éléments de la liste ou items.

Exemples : Liste1=[1,2,56,87,74,3] ou Liste2=["rouge",[45,34],223].

Une liste n'a pas de taille fixée, on peut y mettre ce que l'on veut et on peut modifier, supprimer, ajouter des éléments.

On dit que les tableaux sont **muables** ou **mutables** : ils peuvent être modifiés après leur création. Ils ne peuvent donc pas être une clé dans un dictionnaire.

II. TABLEAUX ET INDICES

Chaque élément d'une liste est repéré par un indice.

Le premier élément d'une liste a pour indice 0. *(comme la première case d'un tableau)*

Exemple :

Liste1[0]= 1....., Liste1[1]= 2....., Liste1[4]= 74.....
 Liste2[0]= 'rouge'....., Liste2[1]= [45,34]....., Liste2[1][0]= 45.....

III. TABLEAUX ET PYTHON

Instruction	Effet
L=[e1,e2,...,en]	Crée une liste L contenant les éléments e1, e2,...,en
L=[]	Crée une liste vide
L[i]	Renvoie l'élément d'indice i de la liste L
L[-1]	Renvoie le dernier élément de la liste L
L[i]=elem	Affecte à l'élément d'indice i l'élément elem
len(L)	Renvoie le nombre d'éléments de la liste L
e in L	Teste si e est dans L

Méthode/Fonction	Effet
L.append(elem)	Ajoute elem à la fin de la liste
L.pop(i)	Supprime l'élément d'indice i de L
L.pop()	Supprime le dernier élément de L
[1,2]+[3,4]	+ : concatène les 2 listes : [1,2,3,4]
[1,2]*3	* : duplique. Donne [1,2,1,2,1,2]
L.sort() ou sorted(L)	Trie la liste L
choice(L)	Choisit au hasard un élément de L (nécessite le module Random)
L[:m]	Affiche les m premiers éléments de L
L[m:n]	Affiche les éléments de l'indice m à l'indice n-1

On pourrait contourner ce problème grâce à `copie=liste[:]`.

```
liste = ['a', 'b', 'c']
copie=liste[:]
print(copie)
liste[0]="modifié"
print(liste)
```

Tout semble bien se passer... Mais que se passe-t-il si l'un des éléments est une liste? On retombe sur le même problème... puisque c'est l'adresse mémoire qui est copiée :

```
liste = ['a', 'b', [0,1]]
copie=liste[:]
liste[0]="modifié"
liste[2][0]="MODIF"
print(liste)
print(copie)
```

Pour contourner la situation, il faut utiliser la méthode `liste.deepcopy()` du module `copy`. Cette méthode effectue une "copie profonde".

La méthode `liste.copy` ne suffit pas non plus.

```
liste = ['a', 'b', [0,1]]
copie=deepcopy(liste)
liste[0]="modifié"
liste[2][0]="MODIF"
print(liste)
print(copie)
```

Des vidéos :

- L'affectation en Python: https://www.youtube.com/watch?v=_jN8PwnqNks
- La copie de listes: <https://www.youtube.com/watch?v=I3dbeflpI-Q>

5. Constructions de tableaux

a. Avec la fonction `range` :

- `liste1=list(range(4))` crée la liste $[0, 1, 2, 3]$... (4 entiers à partir de zéro)...
- `liste2=list(range(3,6))` crée la liste $[3, 4, 5]$... (6-3=3 entiers à partir de 3)...
- `liste3=list(range(3,11,2))` crée la liste $[3, 5, 7, 9]$... (nombres à partir de 3 jusqu'à 11 non inclus, par pas de 2)

b. Construction par compréhension : cette méthode permet de réduire le code

On utilise la syntaxe : `nouv_liste=[<fonction sur item> for <item> in <liste> if <condition>]`

Exemple : `[f(x) for x in liste]` correspond à $\{f(x)/x \in \text{liste}\}$.
liste = [0, 1, 2 ... 9]
liste_carres = [0², 1², 2² ... 9²]

```
liste = range(10)
liste_carres=[x**2 for x in liste]
```

Exemple : `[f(x) for x in liste if Condition(x)]` correspond à $\{f(x)/x \in \text{liste tel que Conditions}(x)\}$.

```
liste = range(10)
liste_carres=[x**2 for x in liste if (x%2==0)]
```

Exemple :

On souhaite ne garder que les notes supérieures à la moyenne dans la liste suivante

$L=[1,2,8,20,15,14,5,4,2,18,7]$. Le code Python sera :

L_moyenne = [x for x in L if (x >= 10)].....

IV. LES TABLEAUX DE TABLEAUX OU MATRICES

Une matrice est un tableau de tableaux de même longueur.

$$\text{Mat} = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]] \quad \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{pmatrix}$$

Pour parcourir une matrice, on utilise des boucles imbriquées :

```
n=len(Mat)
m=len(Mat[0])
for i in range(n):
    for j in range(m):
        print(Mat[i][j])
```

Attention ! Pour initialiser une matrice de valeurs muables indépendantes, on peut procéder de 3 façons :

```
t1=[[0]*2 for i in range(3)]
t1[0][1]=1
print(t1)
t2=[[0,0] for i in range(3)]
t2[0][1]=1
print(t2)
t3=[[0]*2,[0]*2,[0]*2]
t3[0][1]=1
print(t3)
t4=[None]*3
for i in range(3):
    t4[i]=[0]*2
t4[0][1]=1
print(t4)
```

$t_1 = \begin{bmatrix} [0,0] \\ [0,0] \\ [0,0] \end{bmatrix}$
 $t_1 = \begin{bmatrix} [0,1] \\ [0,0] \\ [0,0] \end{bmatrix}$
 $t_2 = \begin{bmatrix} [0,0] \\ [0,0] \\ [0,0] \end{bmatrix}$
 $t_2 = \begin{bmatrix} [0,1] \\ [0,0] \\ [0,0] \end{bmatrix}$
 $t_3 = \begin{bmatrix} [0,0] \\ [0,0] \\ [0,0] \end{bmatrix}$
 $t_3 = \begin{bmatrix} [0,1] \\ [0,0] \\ [0,0] \end{bmatrix}$
 $t_4 = \begin{bmatrix} [0,0] \\ [0,0] \\ [0,0] \end{bmatrix}$
 $t_4 = \begin{bmatrix} [0,1] \\ [0,0] \\ [0,0] \end{bmatrix}$

Cette façon ci-dessous, ne fonctionne pas ! Vérifiez-le...

```
t5=[[0,0]]*3
t5[0][1]=1
print(t5)
```

$t_5 = \begin{bmatrix} [0,0] \\ [0,0] \\ [0,0] \end{bmatrix}$
 $t_5 = \begin{bmatrix} [0,1] \\ [0,1] \\ [0,1] \end{bmatrix}$

Exemple :

	Vendeur 1	Vendeur 2	Vendeur 3
Croissant	4	5	2
Chocolatine	3	6	8

Ce tableau peut-être représenté par une matrice en maths :

$$\begin{pmatrix} 4 & 5 & 2 \\ 3 & 6 & 8 \end{pmatrix}$$

ou par une liste de liste en informatique :

$[[4,5,2],[3,6,8]]$

On a accès aux éléments : $L[1][2] = 8$

V. EXERCICES

1. Vrai/Faux ?

☐ a. $L = [1, 2, 3]$. Après l'instruction $L.append([4, 5])$, $L = [1, 2, 3, 4, 5]$

☒ b. $L = [3*x \text{ for } x \text{ in } [1, 2, 3] \text{ if } x \% 2]$. L a pour valeur $[3, 9]$

☐ c. Soit T un tableau de nombres. Pour modifier le dernier élément, on écrit : $T[\text{len}(T)] = \dots$

$T[\text{len}(T)-1]$
ou $T[-1]$

2. QCM. Pour chaque question, une seule réponse est correcte.

- a. On définit couleur par couleur=["bleu","vert","rouge","marron"]. Alors, l'indice (ou index) de
 i. couleur est 5 ii. "vert" est 2 iii. "marron" est 3.
- b. Si lettres=["a","b","c","c","d"] et si on écrit ensuite lettres.append("c"), que vaut len(lettres) ?
 i. 3 ii. 4 iii. 5 iv. 6.
- c. Pour construire une table de 7 lignes et 7 valeurs remplies de 1, on utilise la syntaxe :
 i. [1]*7 for i in range(7) ii. for i in range(7) [1]*7
iii. [[1]*7 for i in range(7)] iv. [for i in range(7) [1]*7]
- d. L=[15,17,12,23]. Après l'instruction L[2]=25, la liste L a pour valeur :
 i. [15,17,12,23] ii. [15,25,12,23] iii. [15,17,25,23] iv. [15,25,17,12,23]
- e. L=[[1,2,3],[4,5,6],[7,8,9]], la valeur de L[1][2] est : i. 2 ii. 4 iii. 6 iv. 8
- f. Après l'instruction L=[[i,i+1] for i in range(2)], la valeur de L est :
i. [[0,1],[1,2]] ii. [[1,2],[2,3]] iii. [0,1,1,2] iv. [1,2,2,3]
3. Créer une liste en compréhension contenant les cubes des 4 premiers entiers. De même, créer une liste contenant les 4 premiers multiples de 0.2. *liste_cubes = [x**2 for x in range(1,4+1)]*
*liste_multiples = [x*0.2 for x in range(1,4+1)]*
4. a. Écrire un programme qui construit une liste de la forme [1,1,1,2,1,3,1,4,1,5,1,...,1,n].
 b. Écrire un programme qui construit une liste de la forme [1,1,2,1,2,3,1,2,3,4,...,1,2,3,4,...,n]
5. Écrire un programme qui :
 a. Définit une liste contenant les entiers de 1 à 10 et calcule la somme des éléments de la liste.
 b. Calcule et affiche la moyenne de ces nombres.
6. On donne les chaînes de caractères : ch1="abc" et ch2="de".
 Écrire un programme permettant de créer la liste ["ad","ae","bd","be","cd","ce"].
7. Écrire une fonction min_max_moy admettant une liste comme argument et renvoyant le minimum, le maximum et la moyenne des valeurs de la liste.
8. Écrire une fonction nb_occurrences qui recherche le nombre d'occurrences d'un élément dans un tableau.
9. Expliquer la différence entre ces 2 programmes :

```

1 #script 1
2 L=[0,0,0,0]
3 L1=[]
4 for i in range(7):
5     L1.append(L)
6
7 print(L1)
8
9 L1[2][3]=1
10 print(L1)
    
```

→ L déclarée avant la boucle, persistante
 → On ajoute L à L1 7 fois
 → L1 = 7 * L1 = [0,0,0,0] * 7 = [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
 → Mais L1[2][3] = L1[3] donc L = [0,0,0,1]
 → L1 = 7 * L = [0,0,0,1] * 7 = [0,0,0,1,0,0,0,1,0,0,0,1,0,0,0,1,0,0,0,1]

```

1 #script 2
2 L2=[]
3 for i in range(7):
4     L=[0,0,0,0]
5     L2.append(L)
6
7 print(L2)
8
9 L2[2][3]=1
10 print(L2)
    
```

→ L déclarée dans la boucle → éphémère
 → On ajoute [0,0,0,0] à L2 7 fois

10. Écrire une fonction **RechercheMin** qui prend en paramètre un tableau de nombres non trié **tab**, et qui renvoie l'indice de la première occurrence du minimum de ce tableau. Les tableaux seront représentés sous forme de liste Python.

Exemples :

```
>>> indice_du_min([5])
0
>>> indice_du_min([2, 4, 1])
2
>>> indice_du_min([5, 3, 2, 2, 4])
2
```

11. On considère la fonction **separe** ci-dessous qui prend en argument un tableau **tab** dont les éléments sont des 0 et des 1 et qui sépare les 0 des 1 en plaçant les 0 en début de tableau et les 1 à la suite. Compléter la fonction **separe** ci-dessous.

```
def separe(tab):
    i = 0
    j = len(tab) - 1
    while i < j :
        if tab[i] == 0 :
            i = i + 1
        else :
            tab[i], tab[j] = 0, 1
            j = j - 1
    return tab
```

Exemples :

```
>>> separe([1, 0, 1, 0, 1, 0, 1, 0])
[0, 0, 0, 0, 1, 1, 1, 1]
>>> separe([1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0])
[0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

12. Programmer la fonction **verifie** qui prend en paramètre un tableau de valeurs numériques non vide et qui renvoie True si ce tableau est trié dans l'ordre croissant, False sinon.

Exemples :

```
>>> verifie([0, 5, 8, 8, 9])
True
>>> verifie([8, 12, 4])
False
>>> verifie([-1, 4])
True
>>> verifie([5])
True
```

13. On a relevé les valeurs moyennes annuelles des températures à Paris pour la période allant de 2013 à 2019. Les résultats ont été récupérés sous la forme de deux tableaux (de type list) : l'un pour les températures, l'autre pour les années :

```
t_moy = [14.9, 13.3, 13.1, 12.5, 13.0, 13.6, 13.7]
annees = [2013, 2014, 2015, 2016, 2017, 2018, 2019]
```

Écrire la fonction **annee_temperature_minimale** qui prend en paramètres ces deux tableaux et qui renvoie la plus petite valeur relevée au cours de la période et l'année correspondante.

On suppose que la température minimale est atteinte une seule fois.

Exemple :

```
>>> annee_temperature_minimale(t_moy, annees)
(12.5, 2016)
```

14. Exercice type BAC - Sur Capytale : code **c34e-2740285**

Exercice 3

<https://www.sujetdebac.fr/annales-pdf/2021/spe-numerique-informatique-2021-amerique-nord-1-sujet-officiel.pdf>

